XR

HACKATHON CHALLENGE 2026

Innovate. Collaborate. Build.

Call For Participation



Organized by Samsung R&D Institute India, Bangalore

Challenge Tracks

Four exciting problem statements

TRACK 1

Synthetic Eye Movement & Virtual Camera Rendering

Calibrated programmatic eye articulation, fixation convergence, and eye-midpoint rendering in Blender.

TRACK 2

EchoGesture: Sign Language Prediction on Edge Device

Fully on-device, exocentric sign language translation on wearable edge processors.

TRACK 3

Path Finder: Trajectory Estimation Under Sensor Degradation

6-DoF trajectory reconstruction and 5-second future dead reckoning using only degraded IMU telemetry.

TRACK 4

Action Aware Video Summarization

Build a system that understands actions and object interactions in typically 2–10 minutes first-person videos, selects important temporal segments, and compiles a concise highlight from the original footage.

[TRACK 01]

Synthetic Eye Movement & Virtual Camera Rendering

XR Perception Group

About the Problem

Track 01

CORE MOTIVATION: *The Synthetic Gaze Data Gap*

Training robust XR gaze estimators requires massive, ground-truth annotated gaze datasets, yet capturing real human eyes across extreme gaze angles is slow, costly, and restricted by strict privacy regulations.

Anatomical & Kinetic Realism

2D planar pupil translation looks unnatural; realistic gaze simulation requires true 3D spherical eyeball rotation around physical centers.

Rigid Metric Calibration

2D planar pupil translation looks unnatural; realistic gaze simulation requires true 3D spherical eyeball rotation around physical centers.

CHALLENGE ABSTRACT

- **Build:** textured, articulatable face model in Blender; a Python wrapper rotates both eyeballs about their centers — pupils move Left / Right / Up / Down with a center reset.
- **Capture:** standardized 1024×1024 renders from a camera fixed 40 cm ahead.
- **Evaluate:** 5-directional pass/fail check (100 pts) in a 10-day rapid prototyping sprint.

Challenge Blueprint

Track 01

KEY SPECIFICATIONS

- **Textured, articulatable face rig** (MakeHuman / GNM / FLAME) with left/right eye bones
- `Fixation 35 cm` forward; locked camera 40 cm; 1024×1024 PNG output
- Deterministic Python controls: `eyes.move(h, v)` / `eyes.reset()`
- Synchronous pupil articulation + 10 clockwise renders at $\theta k = 2\pi k/10$
- Per-frame ground-truth JSON with 2D pupil centers

REQUIRED DELIVERABLES

- Complete .blend scene — textured mesh, eye bones, materials
- Python wrapper + README with setup instructions
- 10 clockwise renders + per-frame JSON metadata
- Contact sheet or 1–2 min demo video

EVALUATION (100 PTS)

- **Execution & Efficiency — 40%**
  Calibration 20% · Automation 20%
- **Quality & Movement — 60%**
  Direction pass/fail 40% · Sequence & metadata 20%

Submission Guidelines

Track 01

One zipped bundle per team. Evaluators run the wrapper directly — non-standard layouts fail.

SUBMISSION BUNDLE — team_name.zip

team_name.zip

```
team_name.zip
├── scene.blend
│     # textured mesh, eye bones, materials
├── wrapper/eyes.py
│     # eyes.move(h,v) / eyes.reset() / eyes.render()
├── renders/
│     # 10 clockwise frames, θk = 2πk/10
├── metadata.json
│     # per-frame inputs, rotations, pupil centers
└── README.md
      # setup & run instructions
```

SCENE CALIBRATION CONSTRAINTS

- **Fixation target:** 35 cm forward from eye midpoint (motion bounds ± 5 cm)
- **Render camera:** locked 40 cm forward, facing the face
- **Output:** 1024×1024 PNG under constant lighting
- **Clockwise sampling:** $\theta k = 2\pi k/10$ ($k = 0..9$), $h = \sin \theta k$, $v = \cos \theta k$, starting top
- **Demo:** contact sheet or 1–2 min video of the wrapper in action
- **Open assets:** MakeHuman / MPFB (CC0), GNM Head Editor, Textured FLAME

Submission Guidelines & Evaluation

Track 01

PER-FRAME METADATA — metadata.json

metadata.json — one record per render

```
{
  "frame": 3,
  "theta": 1.8849,        // θk = 2πk/10
  "input":  { "h": 0.951, "v": 0.309 },
  "eye_rotation_L": [ rx, ry, rz ],
  "eye_rotation_R": [ rx, ry, rz ],
  "pupil_center_2D_L": [ x, y ],
  "pupil_center_2D_R": [ x, y ],
  "camera_matrix": [ ... ]
}

// normalized inputs [-1, +1]; 3D rotations in
// world frame; 2D pupils projected at 40 cm
```

EVALUATION (100 PTS)

- **Execution & Efficiency — 40%**
- Calibration 20% · Automation 20%
- **Quality & Movement — 60%**
- Direction pass/fail 40% · Sequence & metadata 20%

DIRECTION CHECK (PASS/FAIL)

Pupil shift must be visibly correct vs center for Left, Right, Up, Down and Reset — all 5 of 5 directions must pass.

[TRACK 02]

EchoGesture: Sign Language Prediction on Edge Devices

XR Perception Group

About the Problem

Track 02

CURRENT MOTIVATION: Conversation & Privacy Gaps

Real-time sign language translation on edge devices is bottlenecked by the rapid conversational cadence of natural signing and the strict imperative that bystander video must never leave the device.

The Conversation Gap

Translation must keep pace with ≤ 50 ms latency or dialogue turns awkward and out of sync.

The On-Device Privacy Principle

All visual processing stays on the device — zero raw bystander video uploaded to the cloud.

CHALLENGE ABSTRACT

- **Fully on-device, exocentric** pipeline: live smartphone camera video → translated text or audio.
- Recognizes conversational sign gestures in real time without draining battery or thermal throttling.
- Budgets: `≤ 20 MB` quantized model (INT8), `≤ 50 ms` latency, zero cloud streaming.
- Datasets: How2Sign (80+ h ASL) & WLASL (2,000 words); starter baselines provided.

Challenge Blueprint

Track 02

KEY CONSTRAINTS & ARCHITECTURE

- Exocentric third-person view: moving heads, angled signers, low light
- On-device privacy: strictly local processing, zero cloud streaming
- Pose-invariant 3D hand features; quantized TFLite / ONNX Runtime model
- Captions on display or audio output for two-way conversation

REQUIRED DELIVERABLES

- GitHub repo: `predict()` video → text, reproducible setup
- `predictions.jsonl` (one JSON per clip) + results.csv
- Quantized model + conversion script (< 20 MB INT8)
- Demo video (2–3 min) or deployed app URL

EVALUATION (100 PTS)

- **Execution & Efficiency — 40%**

  Model quantization 20% · Edge latency 20%

- **Quality & Accuracy — 60%**

  Top-1 accuracy 35% · Robustness & HUD 25%

Submission Guidelines

Track 02

1. Prediction output (what the evaluator runs)

Each team implements one function that takes a clip path and returns a JSON object:

Python

```
def predict(video_path: str) -> dict:  """Return the predicted gloss/word for one video clip. Must run fully on-device."""
```

Return format: json

```
{  "clip_id": "test_0412",  "prediction": "HAPPY",  "top5": ["HAPPY", "HAND", "HOUSE", "HELP", "HOME"],  "confidence": 0.81,  "latency_ms": 23.4 }
```

- clip_id must match the test manifest exactly.
- prediction is a word from the fixed vocabulary list, uppercase.
- top5 is optional but lets the organizers compute top-5 accuracy.
- latency_ms is self-reported. The organizers re-measure on their own device.

Teams submit a single predictions.jsonl file, one JSON object per line, one line per test clip. The evaluator computes accuracy and checks that every clip ID is present.

2. Team results sheet (filled in by participants)

A CSV with one row per metric. Each row has a measured value, a target, and a pass/fail column.

metric	target	measured unit	device	runtime	pass
top1_accuracy	>=70	%			
latency_p95	<=50	ms			
model_size	<=25	MB			
accuracy_drop_robustness	<=5	pt			
avg_power	<=500	mW			
thermal_drift_10min	<=20	%			

Teams fill in measured, device (for example, “Galaxy S25, Snapdragon 8 Elite”), and runtime (for example, “LiteRT INT8, GPU delegate”). The pass column can be computed by a short script that compares measured against target.

3. Submission package

- predictions.json and results.csv in results folder
- Repository link with the predict() in main.py, requirements.txt file for dependencies
- Quantized model file and conversion script
- Demo video link (2–3 min) or deployed app URL

EVALUATION CRITERIA
100-POINT JURY MATRIX

1. Execution & Efficiency — 40%

- Model Quantization (20%): Compilation to TFLite/ONNX <20MB.
- Edge Latency (20%): Measured inference time <50ms.

2. Quality & Accuracy — 60%

- Top-1 Accuracy (35%): Precision on held-out test videos.
- Robustness & Delivery (25%): Occlusions & HUD clarity.

Submission Bundle & Evaluation

Track 02

SUBMISSION BUNDLE — team_name.zip

team_name.zip

```
team_name.zip
├── src/
|     # complete source for reproducibility
|     ├── requirements.txt
|     |     # Necessary packages for running
|     ├── main.py
|     |     # with the predict function from the previous slide
|     └── README.md
|           # how to compile & run your code
├── results/
|     ├── predictions.json
|     └── results.csv
```

AUTOMATED EVALUATION

- **Reproducible source code**– the entire implementation lives under `src/`.
- **Exact file-folder layout** – keep the hierarchy shown below; you may add extra helper files or sub-folders as long as the core structure stays unchanged.
- **Python 3.11** – all scripts are written for Python 3.11
- **Model evaluation** – the model is evaluated on a held-out dataset

[TRACK 03]

Path Finder: Trajectory Estimation Under Sensor Degradation

XR Perception Group

About the Problem

Track 03

CORE MOTIVATION: Sensor Jamming & Progressive Degradation

In GPS-denied and hostile environments, an aeroplane navigates through waypoints capturing reconnaissance images. As it proceeds, enemy jamming causes escalating IMU noise, bias drift, and frame dropouts until the IMU

Drift & Frame Dropouts

Double integration explodes quadratically under sensor noise, bias drift, and 2-second signal loss.

Post-Failure Prediction

After the final checkpoint the IMU dies; the aircraft must dead-reckon its 3D path for 5 seconds.

CHALLENGE ABSTRACT

- Estimate the **`6-DoF trajectory`** (position + orientation) through all mission checkpoints using only degraded IMU streams.
- After complete IMU failure, **`dead-reckon the 5-second trajectory`** to strike the target.
- Budgets: **`≤ 8 MB`** peak RAM, **`≤ 20 ms`** compute per 1 s of IMU, single CPU thread, offline.
- Datasets: EuRoC MAV & TUM-VI; organizer degradation script provided.

Challenge Blueprint

Track 03

INPUTS & CONSTRAINTS

- Raw IMU CSV; progressive noise, bias drift, dropouts; unknown gravity alignment
- `≤ 8 MB` peak RAM · `≤ 20 ms` per 1 s of IMU · single CPU thread · offline
- 5-second post-failure trajectory extrapolation (dead reckoning)
- Tiers: T1 checkpoints · T2 blind 5 s (+20%) · T3 dropout resilience (+20%)

REQUIRED DELIVERABLES

- Standalone Python script: degraded IMU CSV → 6-DoF trajectory
- Trajectory CSV: `timestamp, x, y, z, qx, qy, qz, qw`
- Pipeline walkthrough video (2–3 min)
- Datasets: EuRoC MAV & TUM-VI; degradation script provided

EVALUATION (100 PTS)

- **Execution & Efficiency — 40%**
  - Peak memory 20% · Runtime 20%
- **Trajectory Closeness — 60%**
  - Waypoints 30% · Blind 5 s 20% · Dropout 10%

Data Format: Dataset Input (EuRoC IMU)

Track 03

The evaluation dataset is a zipped folder of raw IMU telemetry. Non-conforming submissions fail the automated evaluation pipeline.

INPUT SPECIFICATIONS

- **Zipped folder** with raw IMU telemetry — industry-standard EuRoC MAV format
- **File path:** `imu0/data.csv`
- **Delimiter:** `comma-separated values ( , )`
- **Header line:** `#timestamp [ns], w_RS_S_* [rad s^-1], a_RS_S_* [m s^-2]`
- `timestamp [ns]` — 64-bit integer, nanoseconds since Unix epoch
- `w_RS_S_x / _y / _z` — compensated angular velocity (rad/s)
- `a_RS_S_x / _y / _z` — compensated linear acceleration (m/s^2, includes gravity)

SAMPLE INPUT — imu0/data.csv

```
#timestamp [ns], w_RS_S_x [rad s^-1],
w_RS_S_y [rad s^-1], w_RS_S_z [rad s^-1],
a_RS_S_x [m s^-2], a_RS_S_y [m s^-2],
a_RS_S_z [m s^-2]

1403636579758555392,0.0124,-0.0023,
0.0051,0.1241,0.0312,9.8105
1403636579763555392,0.0119,-0.0021,
0.0049,0.1235,0.0305,9.8099

… one row per IMU sample
```

Data Format: Required Output (TUM Pose)

Track 03

OUTPUT SPECIFICATIONS

- **File name:** `estimated_trajectory.txt` (space-separated)
- Header optional — commented lines must start with `#`
- One estimated pose per row, exactly **8 columns**
- `timestamp` — floating-point seconds (Unix epoch)
- `tx` `ty` `tz` — 3D position vector (meters)
- `qx` `qy` `qz` `qw` — normalized unit quaternion (world frame)

SAMPLE OUTPUT — estimated_trajectory.txt

```
# timestamp tx ty tz qx qy qz qw
1403636579.7585 0.0012 0.0045 0.0102
  0.0001 0.0023 0.0011 0.9999
1403636579.7635 0.0015 0.0049 0.0105
  0.0002 0.0024 0.0011 0.9999

… one pose per IMU frame
```

CRITICAL SYNCHRONIZATION RULES

- **Timestamp conversion:** input is in nanoseconds → output in seconds ($s = ns \times 10^{-9}$).
- **Frequency mapping:** exactly one pose per IMU frame — dropped or misaligned rows incur heavy penalties or failure.

Submission Bundle & Evaluation

Track 03

SUBMISSION BUNDLE — team_name.zip

team_name.zip

```
team_name.zip
├── estimated_trajectory.txt
│     # final generated trajectory file
├── metadata.json
│     # optional: runtime perf, compute specs
├── src/
│     # complete source for reproducibility
│     ├── main.py / main.cpp
│     │     # entrypoint / build system
│     └── README.md
│           # how to compile & run your code
```

AUTOMATED EVALUATION

- Ranked with the open-source **evo** trajectory evaluation suite.
- Your estimated_trajectory.txt is compared against a hidden, high-accuracy ground-truth trajectory.

EVALUATION METRICS

- **ATE (RMSE)** — absolute trajectory error: overall global consistency of your path.
- **RPE** — relative pose error: local drift and rotational error per unit of time over the executed path.

[TRACK 04]

Action-Aware Video Summarization

XR Perception Group

About the Problem

Track 04

OBJECTIVE: Meaningful Highlights from Egocentric Video

Build a system that understands actions and object interactions in typically 2–10 minute first-person videos, selects important temporal segments, and compiles a concise highlight from the original footage.

Action Awareness

Understand meaningful activities and hand–object interactions, not just visual scene changes.

Importance & Redundancy

Retain distinctive events and drop idle, repetitive, or irrelevant footage.

CHALLENGE BRIEF

- **Input:** first-person MP4 (2–10 min); output ≈ 15% highlight compiled from original footage only.
- Output: one `<video_id>.json` per video; run via `run.sh <video.mp4> <out.json>`
- Datasets: EPIC-KITCHENS-100 (138 validation videos) & FPVSum (49 videos).
- XR relevance: first-person frames become thousands of visual tokens — pruning them optimizes downstream XR use cases.

Challenge Blueprint

Track 04

PIPELINE

- Input: first-person MP4 (2–10 min); highlight ≈ 15% of video length
- Action localization + importance ranking; drop idle / redundant footage
- One <video_id>.json per video — same schema across all datasets

REQUIRED DELIVERABLES

- `TEAM_ID/`: run.sh, code/, weights/, requirements.txt, outputs/, README.md
- Validate: `python validate.py --dir outputs/`
- Student kit: manifest.csv, supported_classes.json, validate.py, examples/

EVALUATION (100 PTS)

- Action mAP 25% · Action coverage 40%
- Highlight F1 25% · Label accuracy 10%
- Datasets: EPIC-KITCHENS-100 (138) · FPVSum (49)

Submission Guidelines: Package & Run Cont

Track 04

One run command, one JSON per video, and the same format for every video regardless of its source dataset. Submitted by Oct 28, 2026.

SUBMISSION PACKAGE — TEAM_ID/

TEAM_ID/

```
TEAM_ID/
├── run.sh
│     # <video.mp4> → <out.json>
├── code/
│     # inference code
├── weights/
│     # model weights
├── requirements.txt
├── outputs/
│     # one JSON per video
└── README.md
      # setup & data used
```

VALIDATE BEFORE YOU SUBMIT

- `python validate.py --dir outputs/`
- Checks: all six required schema keys, valid numeric data types and ranges, and strict adherence to all highlight sequence and budget constraints.

STUDENT KIT (SHARED)

- `manifest.csv`, `supported_classes.json`, `validate.py`, a unified `TEAM_README.md`, and sample output JSONs in `examples/`

Submission Guidelines: Output JSON & Eval

Track 04

OUTPUT FORMAT — one <video_id>.json per video

<video_id>.json — same schema for every dataset

```
{
  "video_id": "P01_01",
  "label": "category",        // video-level tag
  "actions": [
    { "verb": "...", "noun": "...",
      "start": 12.4, "end": 15.9 } ],
  "highlights": [
    { "start": 40.2, "end": 58.0,
      "importance": 0.93 } ]
}

// label + actions + highlights; highlights keep
// original footage; aim for ~15% of the video
```

EVALUATION METRICS (100 PTS)

- **Action mAP — 25%**
- Verb–noun overlap with ground truth at tIoU 0.1–0.5.
- **Action coverage — 40%**
- Distinct GT action pairs captured (soft penalty if highlights exceed the 15% budget).
- **Highlight F1 — 25%**
- Frame-level F1 vs top-15% human-annotated consensus importance frames.
- **Label accuracy — 10%**
- Video-level category prediction on FPVSum videos.

Timeline

Key dates



5 NOV

Launch

Themes and problem statements go live. Team Registration opens.



8 NOV

Registration Closes

Final Submission Link Opens



18 NOV

Final Submission closes

Prototype code, 5-min demo video and deck due.



23 NOV

Top 15 announced

Shortlisted teams head to the final demo.



27 NOV

Final demo

Top 15 present idea and solution to the jury.



2 DEC

Final results

Winners announced. Awards and next steps.

** Final Submission Link will be ONLY shared with Teams who have completed the **Team registration process**

Registrations & Submissions

Submission rules

REGISTRATION • Team rules

Google Form

Last date 8 Nov 2026, 11:59 PM

Up to max 4 members
per team, single college

Team name
CollegeName_TeamName

Per member
name, email, phone, year & branch

One theme
one submission per team

FINAL SUBMISSION • Last date 18 Nov 2026, 11:59 PM

Everything below needs to be submitted by 18th Nov 2026. No extension would be provided.

The submission package

- Required deliverable must be as per the selected theme (refer to theme-specific slides)
- Working prototype code — public or shared GitHub repo
- README with reproducible setup instructions, Docker files & other requirements
- Demo video, max 3 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF) — CollegeName_TeamName, follow the nomenclature
- One submission per team via the Google Form

What the presentation covers

- Refer to the CollegeName_TeamName_Submission_ppt
- Theme ID, project title and team details
- Problem statement in your own words
- Solution and architecture diagram
- Tools and tech stack used
- Innovation highlights, results and limitations

**** Teams NOT following the submission guideline would lead to direct disqualification**

What's in it for you

Why take part



Certificates

Every *shortlisted team receives a participation certificate; winners get a merit certificate.



Prizes for the top 4 teams worth ₹1.0 L

Awards for the winning teams at the final results announcement on 2nd Dec 2026



PRISM worklet

**Winning teams get an opportunity to work in PRISM worklet with a Samsung mentor.



Internship at Samsung R&D

Two-month summer internship for ***selected students.

* Shortlisted – Team which will appear for final demo
** Based on the availability of the mentors, students & problem statement
*** Selected students – may have to appear for MAGPIE Coding test

Ready to build?

Registrations open 5 Nov

Registrations open 5th Nov 2026

Pick a track, form a team of up to four, and submit a working prototype with a 5-minute demo video by 18th November.

The top 15 teams present live to a Samsung R&D jury.

Registration link

https://forms.gle/oGBXsydqbhE3HS2o9

Queries

prism@samsung.com

KEY DATES

5 Nov	Launch
8 Nov	Registration closes
18 Nov	Final submission due
23 Nov	Top 15 announced
27 Nov	Final demo round
2 Dec	Final results

Tentative — subject to minor changes.

Organised by the XR team and the PRISM Team, Samsung R&D Institute India, Bangalore

THANK YOU

Build the Future of XR!